

Федеральное государственное автономное образовательное учреждение
высшего образования

**«ОМСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Кафедра Информатика и вычислительная техника.

Лабораторная работа №13

по дисциплине «Программирование» на тему:

«Модульное программирование»

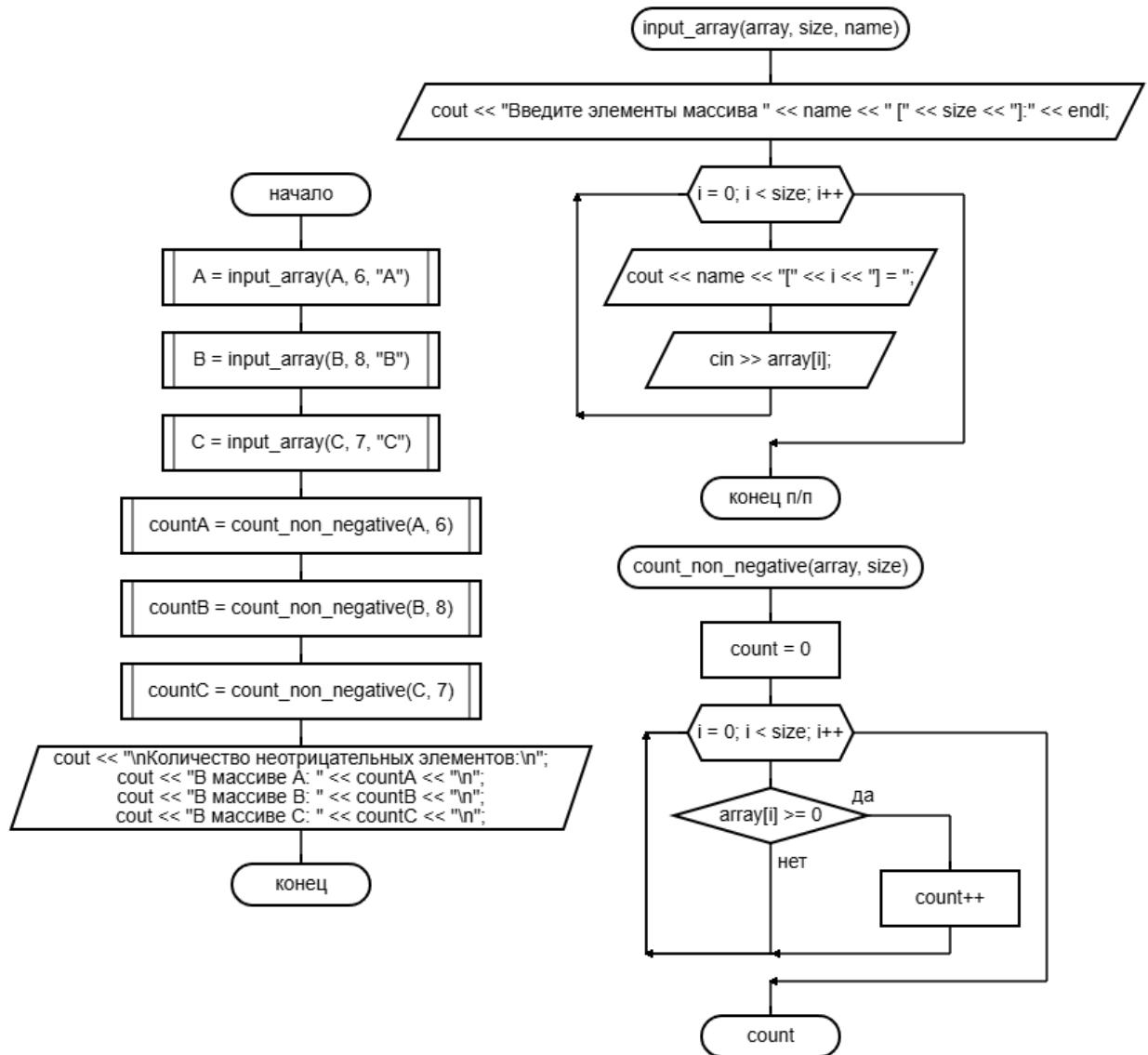
Выполнил: студент группы ИВТ-
244 Шмидт Антон Владиславович
Проверил: ассистент кафедры ИВТ
Симоненко Александр Евгеньевич

Омск 2025

Задача 1 (программа 13_1)

Задача: Для программы 8_3 разработать новую модификацию, скрыв подпрограммы в отдельном файле (модуле), при этом использовать ссылки на переменные, объявленные в другом модуле.

Схема алгоритма:



Решение кодом:

module13_1.cpp

```
#include <iostream>
#include <locale>

using namespace std;

extern int A[6], B[8], C[7];
```

```

void input_array(int array[], int size, const char* name) {
    cout << "Введите элементы массива " << name << " [" << size <<
    "]:" << endl;
    for (int i = 0; i < size; i++) {
        cout << name << "[" << i << "] = ";
        cin >> array[i];
    }
}

int count_non_negative(const int array[], int size) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (array[i] >= 0)
            count++;
    }
    return count;
}

void module13_1() {
    setlocale(LC_ALL, "ru_RU");

    input_array(A, 6, "A");
    input_array(B, 8, "B");
    input_array(C, 7, "C");

    int countA = count_non_negative(A, 6);
    int countB = count_non_negative(B, 8);
    int countC = count_non_negative(C, 7);

    cout << "\nКоличество неотрицательных элементов:\n";
    cout << "В массиве A: " << countA << "\n";
    cout << "В массиве B: " << countB << "\n";
    cout << "В массиве C: " << countC << "\n";
}

```

program13_1.cpp

```

#include <iostream>
#include <locale>

using namespace std;

int A[6], B[8], C[7];

extern void module13_1();

void main13_1() {
    setlocale(LC_ALL, "ru_RU");

    module13_1();
}

```

Результат работы:

Введите элементы массива A [6]:

A[0] = 1

A[1] = 2

A[2] = 3

A[3] = -5

A[4] = 6

A[5] = 7

Введите элементы массива B [8]:

B[0] = -1

B[1] = -2

B[2] = -3

B[3] = -4

B[4] = -5

B[5] = -6

B[6] = -7

B[7] = -8

Введите элементы массива C [7]:

C[0] = 0

C[1] = 0

C[2] = 1

C[3] = 2

C[4] = -4

C[5] = -5

C[6] = 1

Количество неотрицательных элементов:

В массиве A: 5

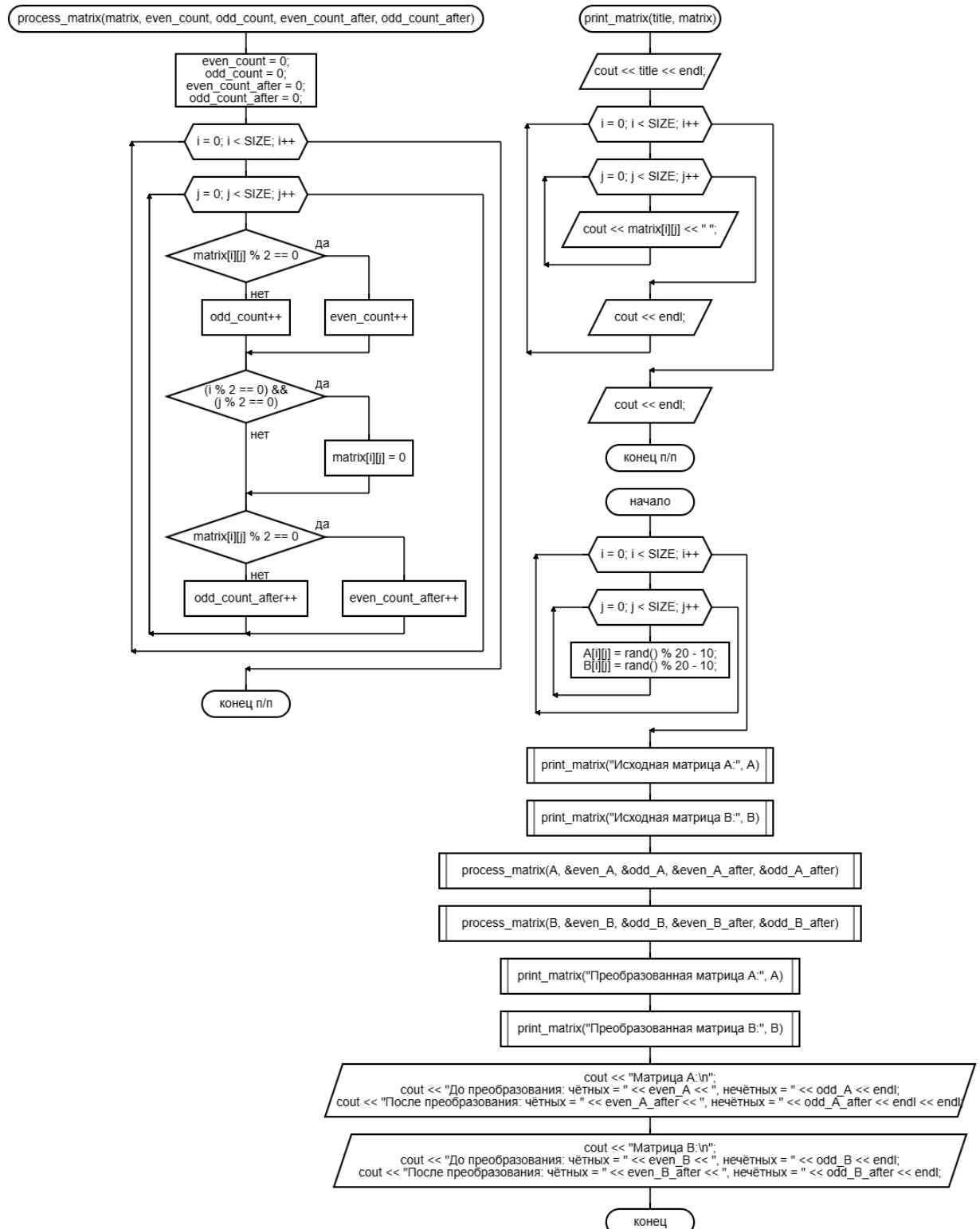
В массиве B: 0

В массиве C: 5

Задача 2 (программа 13_2)

Задача: Для программы 9_2 разработать новую модификацию, скрыв подпрограммы в отдельном файле (модуле), при этом использовать ссылки на переменные, объявленные в другом модуле.

Схема алгоритма:



Решение кодом:

module13_2.cpp

```
#include <iostream>

using namespace std;

const int SIZE = 4;

void process_matrix(int matrix[SIZE][SIZE], int& even_count, int&
odd_count, int& even_count_after, int& odd_count_after) {
    even_count = 0;
    odd_count = 0;
    even_count_after = 0;
    odd_count_after = 0;

    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (matrix[i][j] % 2 == 0) {
                even_count++;
            }
            else {
                odd_count++;
            }

            if ((i % 2 == 0) && (j % 2 == 0)) {
                matrix[i][j] = 0;
            }

            if (matrix[i][j] % 2 == 0) {
                even_count_after++;
            }
            else {
                odd_count_after++;
            }
        }
    }
}

void print_matrix(const char* title, int matrix[SIZE][SIZE]) {
    cout << title << endl;
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
    cout << endl;
}

program13_2.cpp
```

```

#include <iostream>
#include <locale>
#include <ctime>

using namespace std;

const int SIZE = 4;

extern void process_matrix(int matrix[SIZE][SIZE], int&
even_count, int& odd_count, int& even_count_after, int&
odd_count_after);
extern void print_matrix(const char* title, int
matrix[SIZE][SIZE]);

void main13_2() {
    setlocale(LC_ALL, "ru_RU");
    srand(time(NULL));

    int A[SIZE][SIZE], B[SIZE][SIZE];
    int even_A, odd_A, even_A_after, odd_A_after;
    int even_B, odd_B, even_B_after, odd_B_after;

    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            A[i][j] = rand() % 20 - 10;
            B[i][j] = rand() % 20 - 10;
        }
    }

    print_matrix("Исходная матрица A:", A);
    print_matrix("Исходная матрица B:", B);

    process_matrix(A, even_A, odd_A, even_A_after, odd_A_after);
    process_matrix(B, even_B, odd_B, even_B_after, odd_B_after);

    print_matrix("Преобразованная матрица A:", A);
    print_matrix("Преобразованная матрица B:", B);

    cout << "Матрица A:\n";
    cout << "До преобразования: чётных = " << even_A << ",
нечётных = " << odd_A << endl;
    cout << "После преобразования: чётных = " << even_A_after <<
", нечётных = " << odd_A_after << endl << endl;

    cout << "Матрица B:\n";
    cout << "До преобразования: чётных = " << even_B << ",
нечётных = " << odd_B << endl;
    cout << "После преобразования: чётных = " << even_B_after <<
", нечётных = " << odd_B_after << endl;
}

```

Результат работы:

Исходная матрица A:

```
6 3 0 3
4 2 3 -1
-5 5 4 -5
-7 1 7 9
```

Исходная матрица B:

```
-1 -6 -5 -4
-3 -9 6 4
5 2 3 -8
-6 -7 2 -2
```

Преобразованная матрица A:

```
0 3 0 3
4 2 3 -1
0 5 0 -5
-7 1 7 9
```

Преобразованная матрица B:

```
0 -6 0 -4
-3 -9 6 4
0 2 0 -8
-6 -7 2 -2
```

Матрица A:

До преобразования: чётных = 5, нечётных = 11

После преобразования: чётных = 6, нечётных = 10

Матрица B:

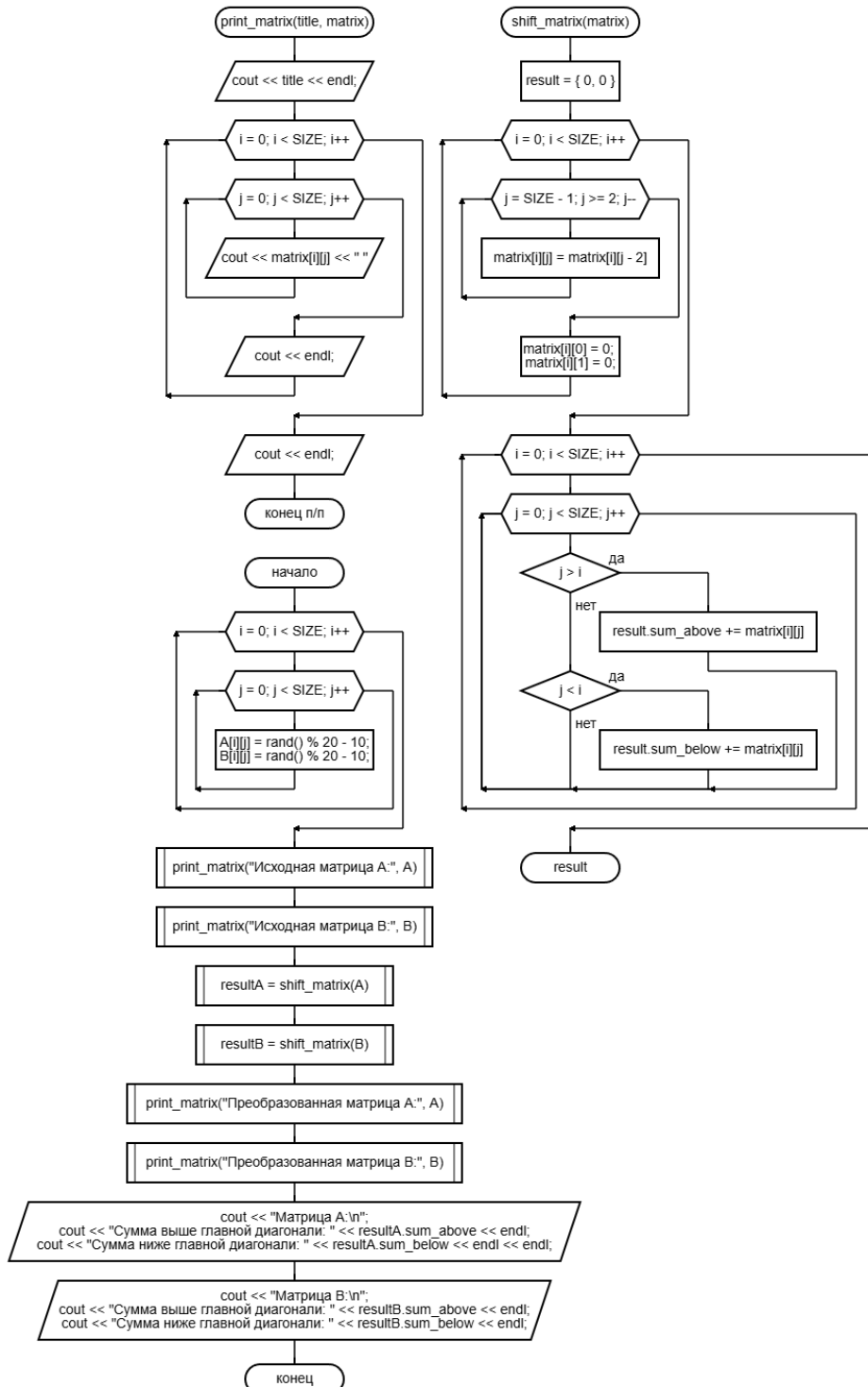
До преобразования: чётных = 9, нечётных = 7

После преобразования: чётных = 13, нечётных = 3

Задача 3 (программа 13_3)

Задача: даны массивы A[4][4], B[4][4]. Получить новые массивы путем сдвига элементов в массивах на два разряда вправо, освободившиеся слева элементы обнулить. Сдвиг элементов в массиве оформить подпрограммой, из подпрограммы вернуть суммы элементов выше главной диагонали и отдельно ниже.

Схема алгоритма:



Решение кодом:

module13_3.cpp

```
#include <iostream>

using namespace std;

const int SIZE = 4;

struct ShiftResult {
    int sum_above;
    int sum_below;
};

ShiftResult shift_matrix(int matrix[SIZE][SIZE]) {
    ShiftResult result = { 0, 0 };

    for (int i = 0; i < SIZE; i++) {
        for (int j = SIZE - 1; j >= 2; j--) {
            matrix[i][j] = matrix[i][j - 2];
        }
        matrix[i][0] = 0;
        matrix[i][1] = 0;
    }

    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (j > i) {
                result.sum_above += matrix[i][j];
            }
            else if (j < i) {
                result.sum_below += matrix[i][j];
            }
        }
    }

    return result;
}

void print_matrix(const char* title, int matrix[SIZE][SIZE]) {
    cout << title << endl;
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
    cout << endl;
}

program13_3.cpp
```

```

#include <iostream>
#include <locale>
#include <ctime>

using namespace std;

const int SIZE = 4;

struct ShiftResult {
    int sum_above;
    int sum_below;
};

extern ShiftResult shift_matrix(int matrix[SIZE][SIZE]);
extern void print_matrix(const char* title, int
matrix[SIZE][SIZE]);

void main13_3() {
    setlocale(LC_ALL, "ru_RU");
    srand(time(NULL));

    int A[SIZE][SIZE], B[SIZE][SIZE];

    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            A[i][j] = rand() % 20 - 10;
            B[i][j] = rand() % 20 - 10;
        }
    }

    print_matrix("Исходная матрица A:", A);
    print_matrix("Исходная матрица B:", B);

    ShiftResult resultA = shift_matrix(A);
    ShiftResult resultB = shift_matrix(B);

    print_matrix("Преобразованная матрица A:", A);
    print_matrix("Преобразованная матрица B:", B);

    cout << "Матрица A:\n";
    cout << "Сумма выше главной диагонали: " << resultA.sum_above
<< endl;
    cout << "Сумма ниже главной диагонали: " << resultA.sum_below
<< endl << endl;

    cout << "Матрица B:\n";
    cout << "Сумма выше главной диагонали: " << resultB.sum_above
<< endl;
    cout << "Сумма ниже главной диагонали: " << resultB.sum_below
<< endl;
}

```

Результат работы:

Исходная матрица A:

```
-5 -2 4 8
1 4 0 6
4 6 1 8
-3 -5 -9 -7
```

Исходная матрица B:

```
2 -4 -2 1
2 2 7 -1
-3 0 -7 -10
-4 9 -8 -10
```

Преобразованная матрица A:

```
0 0 -5 -2
0 0 1 4
0 0 4 6
0 0 -3 -5
```

Преобразованная матрица B:

```
0 0 2 -4
0 0 2 2
0 0 -3 0
0 0 -4 9
```

Матрица A:

Сумма выше главной диагонали: 4
Сумма ниже главной диагонали: -3

Матрица B:

Сумма выше главной диагонали: 2
Сумма ниже главной диагонали: -4